📋 Assignment-7: Implementation of a Multithreaded Number-Processing Pipeline with Synchronization

⚠️ **Important:**

If you notice any mistakes or oversights in this assignment specification document or in any of the provided framework files, please report them to me at the earliest opportunity. I will be more than happy to correct any issues and release updated versions to avoid potential autograding-related problems.

- **⚠️ Although unlimited submissions are allowed before the due date, any submissions or resubmissions after the deadline will strictly not be accepted for grading.**

- **You must test your implementation using the provided testing framework on the Ocelot server before your final submission.**

👬 Team Formation and Submission Guidelines

- Working in a team is optional — you may choose to work individually.
- ⚠️ Each group may include up to **THREE STUDENTS Exceeding this limit will result in a ZERO grade for the entire team.**
- ⚠️ Please ensure that you select your team members **only from within your own section**, and **not from another cross-listed section** I teach, as clearly stated in the syllabus.
  For example:
  - If you are enrolled in **COP4610-RVC**, do **not** form a team with students from **COP4610-RVD**.
  - If you are enrolled in **COP4610-U01**, do **not** form a team with students from **COP4610-U02**.

  ⚠️ Submissions that violate this requirement **will not be graded**, in order to prevent incorrect grade entries.
- Team members **can vary across different assignments** and do not need to remain the same throughout the semester. This guideline is also specified in the course syllabus.
- Only **one team member should submit** the assignment. Including a `README.txt` **file with accurate team member details** is essential for proper grading.
- ⚠️ **If multiple team members submit individually, the assignment will be considered void!** However, each group may make **unlimited submissions before the due date**.

🔑 Table of Contents

1. 📧 TA Contact Information for Your Section
2. 📖 Assignment Overview
3. 🎯 Learning Objectives
4. ⚙️ Assignment Tasks
5. 🗂️ Project Framework
6. 🛠️ Developing and Testing Your Implementation
7. 📤 Assignment Submission Guidelines
8. 📦 Deliverable ZIP file Structure
9. 💯 Grading Rubric

> ⚠️ **Important:** This homework focuses on implementing a parallel number processing pipeline using the producer-consumer pattern with bounded buffers. Students will use pthread mutex and counting semaphores to synchronize multiple generator threads, processor threads, and a writer thread.
>
> All the necessary concepts required to complete this homework are covered under 📦 **Module-4: Process Synchronization & Deadlock Management**. Please review the provided study resources and the textbook chapter contents on the required topics.
>
> **You must use these strict gcc compilation flags while building your application:**
> **-std=c17**
> **-D_POSIX_C_SOURCE=200809L**
> **-Wall**
> **-Wextra**
> **-Werror**
> **-g**
> **-O0**
>
> **You may submit the assignment an unlimited number of times without penalty before the "due date," and with a penalty before the "until date" on the Canvas."**
>
> ⚠️ **Submissions after the "until date" and any resubmissions beyond that point are strictly not allowed!**.
>
> **You must test your implementation with the provided testing framework on the course-dedicated Ocelot server before the final submission.**

---

1. 📧 TA Contact Information for Your Section

For any questions or concerns related to assignment grading, please reach out to the TA listed below.

Communication Guidelines:

- Ensure that all communication is polite and respectful

- **You must contact the Graders first for any grading issues (⚠️ via the Canvas inbox only)**
- ** If the matter remains unresolved, feel free to reach out to the instructor*

Name: Sanskar Lohani
Email: sloha002@fiu.edu
Section: COP4610-UHA

Name: Srushti Visweswaraiah
Email: svisw003@fiu.edu
Sections: COP4610-RVC & COP4610-RVD

Name: Angie Martinez
Email: amart1070@fiu.edu
Sections: COP4610-U01 & COP4610-RVF

---

2. 📖 Assignment Overview

To implement a multi-threaded number processing pipeline that coordinates multiple threads using mutexes and semaphores to safely share data across bounded buffers, ensuring correct ordering and synchronization at every stage.

This assignment involves implementing a **pipeline architecture** with three stages:

1. **Generator threads** (producers) that read from a shared input array and insert numbers into a bounded buffer (**Buffer1**).
2. **Processor threads** (intermediate producers/consumers) that remove numbers from **Buffer1**, square them, and place the results into another bounded buffer (**Buffer2**).
3. A **Writer thread** (final consumer) that removes squared results from **Buffer2** and prints them in the correct input order using a holding buffer and sequence logic.

Each stage is fully parallelized and synchronized using **POSIX semaphores and mutexes**. Correct use of synchronization primitives is essential to avoid data races, deadlocks, and incorrect outputs.

Your implementation will be tested for:

- Correct functional output
- Synchronization correctness (no race conditions or deadlocks)
- Proper thread partitioning and data flow
- Ordered final output (based on input index)

---

3. 🎯 Learning Objectives

By completing this assignment, students will be able to:

- **Explain and implement** the Producer–Consumer pattern using a bounded buffer.
- **Create and manage** multiple POSIX threads (`pthreads`) to perform concurrent producer and consumer tasks.
- **Apply** mutex locks to enforce mutual exclusion during shared buffer access.
- **Use** counting semaphores to correctly track buffer state, including empty and full slots.
- **Coordinate** thread communication using appropriate synchronization primitives to ensure correct execution order.
- **Enforce** deterministic and ordered output across asynchronously executing threads.
- **Detect and avoid** common concurrency issues, including race conditions, deadlocks (particularly due to incorrect mutex–semaphore ordering), and buffer overflows or underflows.
- **Design and implement** thread-safe, modular C programs using `pthreads` and shared data structures.

---

4. ⚙️ Assignment Tasks

This assignment requires you to implement a multi-threaded number processing pipeline using synchronization primitives. You will simulate a pipeline with three fixed stages: **Generators** → **Processors** → **Writer**, with data passed through two bounded buffers.

The main idea is to implement correct inter-thread coordination using **mutexes** and **semaphores** to ensure:

- Proper buffer access (no overflows or underflows)
- Mutual exclusion in critical sections
- Ordering of output based on input position (sequence ID)

Conceptual Workflow

- **Generators (3 threads)**:
  - Divide input numbers among themselves (each thread gets ~N/3 numbers)
  - Each thread inserts `NumberData` structs (number + ID) into BUFFER1
- **Processors (3 threads)**:
  - Consume from BUFFER1
  - Compute `number * number` (square)
  - Insert result as `NumberData` (squared value + same ID) into BUFFER2
  - When finished, signal completion using `sem_post(&processor_done)`

- **Writer (1 thread)**:
  - Collect results from BUFFER2
  - **Buffer out-of-order entries internally**
  - Print results to `stdout` **in input ID order**
  - Waits for all 3 processor threads to finish (via 3 `sem_wait(&processor_done)`)

System Architecture

```
┌──────────────────────────────────────────────────────────────┐
│  GENERATOR THREADS (3 threads - FIXED)                       │
│  Role: Producers for BUFFER1                                 │
│  Task: Generate numbers 1, 2, 3, ..., N                      │
│        Each generator gets N/3 numbers to generate           │
└──────────────────────────────────────────────────────────────┘
                              ↓
┌──────────────────────────────────────────────────────────────┐
│  BUFFER1 (Bounded Buffer)                                    │
│  Capacity: 10 slots                                          │
│  Each slot: NumberData struct (number + id)                  │
│  Synchronization:                                            │
│    - sem_t empty1   // Counting semaphore (init: 10)         │
│    - sem_t full1    // Counting semaphore (init: 0)          │
│    - pthread_mutex_t mutex1 // Binary semaphore              │
└──────────────────────────────────────────────────────────────┘
                              ↓
┌──────────────────────────────────────────────────────────────┐
│  PROCESSOR THREADS (3 threads - FIXED)                       │
│  Role: Consumers of BUFFER1, Producers for BUFFER2           │
│  Task:                                                       │
│    1. Remove number from BUFFER1 (consumer)                  │
│    2. Apply operation: result = number * number (square)     │
│    3. Add result to BUFFER2 (producer)                       │
└──────────────────────────────────────────────────────────────┘
                              ↓
┌──────────────────────────────────────────────────────────────┐
│  BUFFER2 (Bounded Buffer)                                    │
│  Capacity: 10 slots                                          │
│  Each slot: NumberData struct (result + id)                  │
│  Synchronization:                                            │
│    - sem_t empty2   // Counting semaphore (init: 10)         │
│    - pthread_mutex_t mutex2 // Binary semaphore              │
└──────────────────────────────────────────────────────────────┘
                              ↓
┌──────────────────────────────────────────────────────────────┐
│  WRITER THREAD (1 thread - FIXED)                            │
│  Role: Consumer for BUFFER2                                  │
│  Task: Remove results from BUFFER2 → write to output file    │
```

```
│        Must write results in CORRECT ORDER (by id)              │
└──────────────────────────────────────────────────────────────────┘
                                ↓
                  Output File: results.txt

┌──────────────────────────────────────────────────────────────────┐
│  COMPLETION SIGNALING                                            │
│  Semaphore: processor_done (init: 0)                             │
│  - Each processor: sem_post(&processor_done) when done           │
│  - Writer: sem_wait(&processor_done) 3 times                     │
└──────────────────────────────────────────────────────────────────┘
```

⚠️ CRITICAL RULES:

1. **ALWAYS** use `sem_wait` BEFORE `mutex_lock` (prevents deadlock)
2. **ALWAYS** use `mutex_unlock` BEFORE `sem_post` (free-up resource)
3. **ALWAYS** use circular indexing: `(index + 1) % BUFFER_SIZE` (circular queue nature)

Example Execution

Input:

```
./pipeline 10 output.txt
# Process numbers 1 through 10
```

Internal Flow:

```
Generator 0: generates 1, 2, 3, 4    → BUFFER1
Generator 1: generates 5, 6, 7       → BUFFER1
Generator 2: generates 8, 9, 10      → BUFFER1

Processor 0: gets 1 → squares → 1    → BUFFER2
Processor 1: gets 8 → squares → 64   → BUFFER2
Processor 2: gets 5 → squares → 25   → BUFFER2
Processor 0: gets 9 → squares → 81   → BUFFER2
... (order varies due to parallelism)

Writer: writes in ORDER by id:
  1 → 1
  2 → 4
  3 → 9
  4 → 16
  5 → 25
  ...
  10 → 100
```

Output File (results.txt):

```
Input: 1, Result: 1
Input: 2, Result: 4
Input: 3, Result: 9
Input: 4, Result: 16
Input: 5, Result: 25
Input: 6, Result: 36
Input: 7, Result: 49
Input: 8, Result: 64
Input: 9, Result: 81
Input: 10, Result: 100
```

---

5. 📂 Project Framework

This assignment provides a **complete development framework** including **skeleton file** (*template for your implementation*), **driver program**, **header file**, **Makefile**, **testcases** , and **autograder scripts** to guide your implementation process.

Folder Structure

```
PROVIDED_FILES/
├── pipeline.h                  # Header file (Do not modify!)
├── pipeline_driver.c           # Main driver program (Do not modify!)
├── Makefile                    # Makefile (Do not modify!)
├── autograder_pipeline.sh      # Testing script (Do not modify!)
├── batchgrader_pipeline.sh     # Batch testing script (Do not modify!)
├── Skeleton_file(s)/   # Contains the template file(s) for the function(s) you will
implement
└── Testing/                    # Contains the testcases and their corresponding
                                # expected outputs that you are required to PASS

# Detailed homework specification & requirement document
# README.txt - Submit details here in the required format
```

Framework Files (🔒 Do NOT modify these files)

- `pipeline.h`: Complete header file with data structures, constants, and function prototypes. **(study it thoroughly!)**
- `pipeline_driver.c`: The main driver program responsible for coordinating all tests and managing the overall execution flow
- `autograder_pipeline.sh`: Single-submission automated testing script (identical to final grading mechanism)
- `batchgrader_pipeline.sh`: Batch grading script for processing multiple student submissions

- `Makefile`: Used to build your application and generate the final executable. You can use the following make commands:

```
# 📖 To see all available make commands and usage information
make help

# 🏗️ To build your application
make

# 🧹 To clean the build environment (remove old binaries, object files, etc.)
make clean

# 🔄 To rebuild everything from scratch (clean + build)
make rebuild
```

Skeleton Files (✅ You must implement this)

- ⚠️ This template file may include detailed instructions, hints, function signatures, and other information required for your implementation. You are required to strictly adhere to the contents of this file.
  - `skeleton_pipeline.c` → Rename it to `pipeline.c`.

Test Cases

You are provided with **comprehensive testcase sets** to validate your implementation incrementally.

⚠️ **IMPORTANT:**
If your implementation passes the testcase with **100%** accuracy, ***you are likely to receive full credit for your submission, based on the functionality you have implemented***.

Execution Permissions Notice

If you **do not** have execute permissions for the instructor-provided files, you can manually grant the required permissions using the following command on a **Linux system**.

> **Note:** When files are uploaded or transferred to Ocelot server (`ocelot-bbhatkal.aul.fiu.edu`), execute permissions **may be stripped** due to security restrictions. In such cases, you must explicitly grant **read (r)**, **write (w)**, and **execute (x)** permissions.

Granting Permissions to All Items in the Current Directory

```
# Check current permissions
ls -l

# Grant read, write, and execute permissions (safe and preferred – applies only
what's missing)
chmod +rwx *

# Assign full permissions (read, write, execute) to all users for all files in the
current directory
# ⚠️ Not preferred – use only as a last resort!
chmod 777 *

# Verify that permissions were applied
ls -l
```

---

6. 🛠️ Developing and Testing Your Implementation

⚠️ **Important**: **The instructor will use the *same autograder and batchgrader scipts* for final grading. You are required to test your implementation using both the autograder and the batch grader on the `Ocelot server`. Failure to adhere to this guideline will *highly likely result in your submission failing* when graded by the instructor.**

These autograders provide detailed insight into:

- **Code errors**
- **Logic errors**
- **Missing files**
- **Incorrect file names**
- **Any issues that cause the autograder or batch grader to fail**

Testing your work thoroughly with both tools is essential to ensure that your submission is complete, correct, and compatible with the grading system.

Data Structures

You will work with these structures defined in `pipeline.h`:

`NumberData`:

```
typedef struct {
    int number;      // The actual number (input or result)
    int id;          // Sequence number (0, 1, 2, ... N-1)
} NumberData;
```

`BoundedBuffer`:

```
typedef struct {
    NumberData buffer[BUFFER_SIZE];  // 10 slots
    int in;                          // Producer index
    int out;                         // Consumer index
    sem_t empty;                     // Counting semaphore: empty slots
    sem_t full;                      // Counting semaphore: full slots
    pthread_mutex_t mutex;           // Mutex: buffer protection
} BoundedBuffer;
```

✅ Your Task:

You must implement these functions in `pipeline.c`.

- 💡 **Refer to the detailed TODO comments in the skeleton code for specific implementation tasks**.

1. `void init_buffer(BoundedBuffer *buf, int capacity);`
2. `void buffer_add(BoundedBuffer *buf, NumberData data);`
3. `NumberData buffer_remove(BoundedBuffer *buf);`
4. `void destroy_buffer(BoundedBuffer *buf);`

🔧 Testing Your Implementation

Step 1: Before You Start Testing

Ensure that **all** of the following items are placed in **one directory**:

1. **All framework files**
   (Header file, Driver file, Makefile, Autograder, Batchgrader)
2. **Testing folder**
   `Testing/` - containing test case **and** corresponding expected output
3. **Your `.c` implementation file**
4. **A ZIP file** containing:
   - Your submission `.c` file
   - The required `README.txt` file

Step 2: Compile and Test with the Testcase (manual testing)

```
# ⚠️ Test with Simple Test Case

# Compile your implementation and build the executable using the Makefile
make rebuild
```

```
# Run with testcase and redirect the output to the student output file
./pipeline Testing/Testcases/input1.txt >  STUDENT_OUTPUT.txt

# Compare the expected output with your program's output
# If you see NO output from the diff command, your output matches 100%.
# Congratulations – you may now proceed to test with the remaining testcases.
diff -BZ STUDENT_OUTPUT.txt Testing/Expected_Output/output1.txt

# Repeat the above steps to test with remaining testcases
```

Step 3: Use Autograder

```
# Make autograder executable (if needed)
chmod +x autograder_pipeline.sh

# Run the autograder (tests single submission)
./autograder_pipeline.sh
```

Step 4: Use Batchgrader for Final Validation

⚠️ **Important:**
The batchgrader will delete all your implementation `.c` files from the current directory for safety and to ensure a clean environment for every run. You must keep a backup of all your submission `.c` files.

```
# Make batchgrader executable (if needed)
chmod +x batchgrader_pipeline.sh

# Run the batchgrader (tests single submission)
# ⚠️ You must have your submission ZIP file in the current directory
./batchgrader_pipeline.sh
```

☑️ YOU MUST!

- ⚠️ **Test on ocelot server**: `ocelot-bbhatkal.aul.fiu.edu`
- ⚠️ **Test thoroughly before submission** - no excuses accepted
- ⚠️ **"Works on my computer" is NOT accepted** as an excuse
- ✅ **If you pass all test cases on the server, you will likely pass instructor's final grading**
- ✅ **What you see is what you get** - autograder results predict your final grade

---

7. 📤 Assignment Submission Guidelines

Submission Format

- All programming Assignments **must be submitted on Canvas** by the deadline as a **single** `.zip` **file**.
  ❌ *Submissions by any other method will not be accepted.*
- **Filename format:** `A7_FirstName_LastName.zip`
  Example: **A7_Harry_Potter.zip**

⚠️ Ocelot Server Requirement

Your code **must compile and run** on the course-designated Ocelot server:

- 🛑 **If your submission does not compile or run on Ocelot, the grade will be ZERO.**
- This requirement applies to **all** programming homeworks.

⏰ Late Submission Policy

- A **10% penalty per day** applies from the **Due Date** until the **Until Date**.
- 🚫 **No submissions are accepted after the Until Date.**

Additional Information

- Refer to the **course syllabus** for any remaining policies or clarifications.

---

8. 📦 Deliverable ZIP file Structure

> ⚠️ **DO NOT submit any other files other than the required files. The batchgrader may treat any additional items in the ZIP file as invalid, which will result in a grade of zero.**
>
> 📁 All other required framework files and the testcases will be supplied by the instructor to the autograder during final grading.

- You are required to submit your work as **a single ZIP archive file** to the Canvas.
  - **Filename Format**: your `A7_Firstname_Lastname.zip` (**exact format required**).
- **For team submissions** – Include the first name and last name of the team's submitting member in the ZIP file name (e.g., `A7_Harry_Potter.zip`). **For solo submissions** – Use your own first and last name.

```
A7_Harry_Potter.zip
├── pipeline.c        # Your implementation
└── README.txt        # Your details and brief implementation approach
```

---

9. 💯 Grading Rubric

Criteria	Points
⚠️ Program fails to run or crashes during testing	**0**
⚠️ The autograder or batchgrader fail due to the absence of `README.txt`, the required C implementation file(s), or any other required file— ***or for any reason***	**0**
⚠️ Partial Implementation *(score determined solely by autograder results)*	**0 - 90**

Detailed Breakdown of 100 Points

Autograding Component	Points
Functionality & Correctness Tests (each testcase 25 points)	75
Synchronization Tests (Deadlock detection 8 points + Race condition detection 7 points)	15
AUTOGRADER TOTAL	**90**
Manual Grading: Clear code structure, Meaningful comments, well-written README, Overall readability, Following the submission guidelines	10
FINAL TOTAL	**100**

⚠️ **Note**: *The autograder awards 90 points automatically based on correctness. The remaining 10 points are manually graded for code quality, comments, structure, and README completeness.*

- **You are solely responsible** for ensuring that your submission passes all test cases using both the autograder and the batch grader on the course-designated Ocelot server environment. Failure to do so may result in a **zero grade**.
- All final grading is performed on the **Ocelot server** using the provided autograders.
- Claims such as *"it works on my computer but not on yours"* will **not** be accepted.

Good luck! 💪